Zensar Content Studio

Project Documentation

Contents

1. Purpose
2. The three tools
3. Architecture at a glance
4. Technology
5. Privacy and trust
6. Folders
7. Installation and deployment
8. Quality
9. Related documents

1. Purpose

Teams spend a lot of time turning documents into presentations, videos, publisher-ready papers and branded white papers. Zensar Content Studio does this automatically, with three rules:

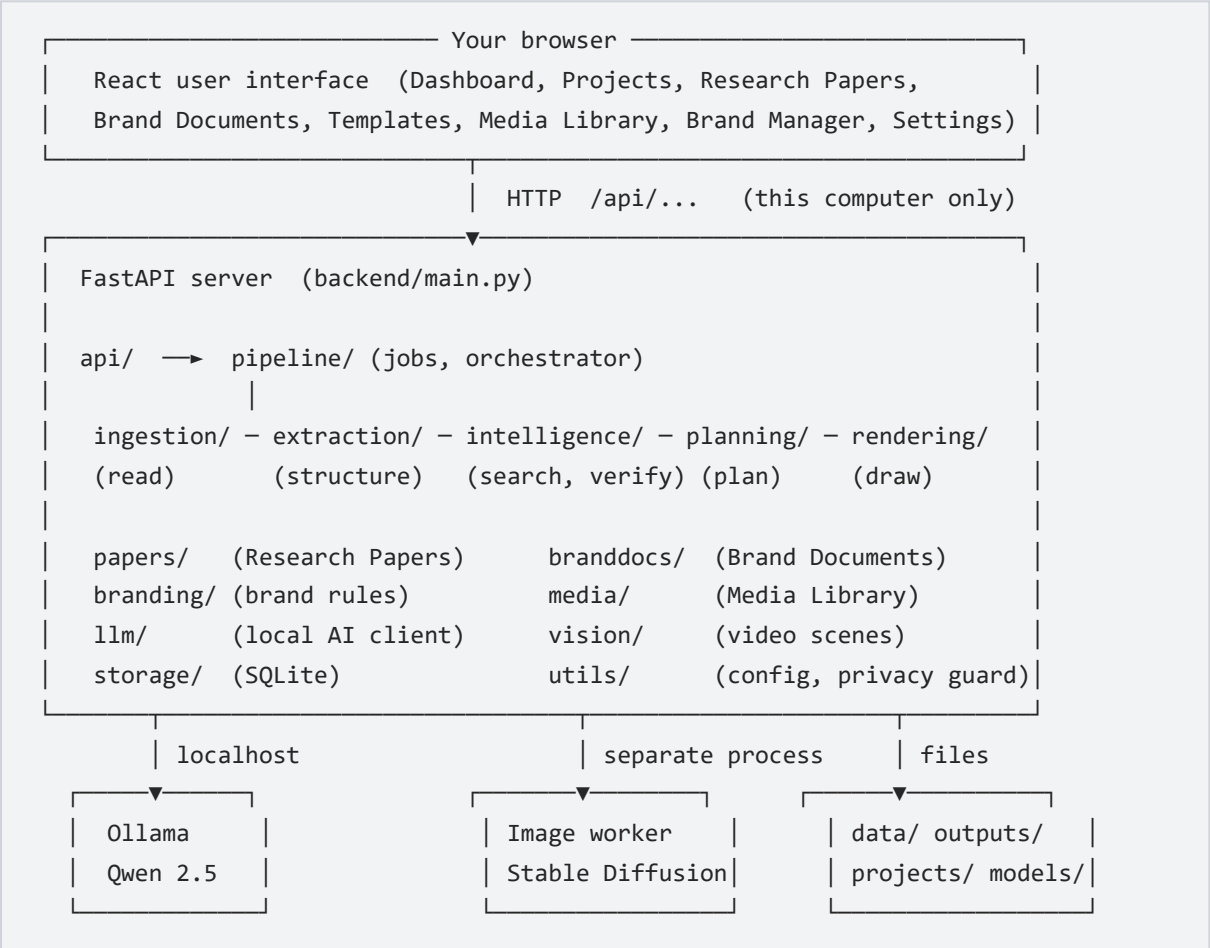
1. **Private:** everything runs on the user's own computer. No document is ever sent to a cloud service.
2. **Faithful:** every statement in the output is traced back to the source. Numbers are checked, and nothing is invented.
3. **On-brand:** the design comes from the brand's own official material, never from guesses.

2. The three tools

Tool	Input	Output
Projects	Documents (PDF, Word, PowerPoint, text, web pages), demo videos, code repositories	Editable PowerPoint, PDF, brand-styled images and diagrams, a narrated MP4 video

Tool	Input	Output
Research Papers	A scientific paper (PDF, Word, LaTeX, Markdown, text)	The paper re-formatted for **Springer Nature, Springer LNCS, Elsevier, IEEE, ACM or APA 7**: LaTeX project, Word, PDF, BibTeX and a checks report
Brand Documents	Any document (PDF, Word, Markdown, text) plus a branded **reference PDF**	A branded white paper (PDF) laid out like the reference, fitted to its length, with images created for the content

3. Architecture at a glance



- The **interface** only displays things and sends requests. All the work happens in the **server**.
- Long tasks run as **background jobs**. The interface checks their progress every couple of seconds.
- The **writing AI** (Ollama) and the **image AI** (Stable Diffusion) run as local programs. The image AI runs in its own process, so its memory is freed after each job.
- A **network guard** blocks every outgoing connection except to this computer.

4. Technology

Area	Technology	Used for
Interface	React 18, TypeScript, Tailwind CSS 4, Vite 6, lucide icons	Screens in the browser
Server	Python 3.10–3.12, FastAPI, Uvicorn, Pydantic	API, background jobs, data checking
Storage	SQLite + files	Projects, jobs, settings, outputs
Reading documents	PyMuPDF, python-docx, python-pptx, BeautifulSoup, lxml, RapidOCR (optional)	PDF, Word, PowerPoint, HTML, scanned pages
Meaning search	all-MiniLM-L6-v2 (ONNX Runtime + tokenizers), FAISS	Finding relevant passages
Writing AI	**Qwen 2.5** (7B or 3B) via **Ollama**	Planning, condensing, scene descriptions
Image AI	**DreamShaper 8 LCM** (Stable Diffusion 1.5) via Diffusers + PyTorch	Pictures for brand documents
Vision AI	qwen2.5vl:3b via Ollama (optional)	Describing demo-video scenes
Speech	Windows SAPI voices (optional Piper)	Video narration
Output	python-pptx, ReportLab, Pillow, Matplotlib, FFmpeg (imageio-ffmpeg), fontTools	PowerPoint, PDF, images, charts, video, fonts
Papers	citeproc-py + official CSL styles, bibtexparser, pylatexenc, Jinja2	Citations, references, LaTeX
Testing	pytest (127 tests)	Automatic quality checks
Setup	`start.bat` → PowerShell `bootstrap.ps1`	One-click install and start

AI models

Model	Size	Licence	Runs on
Qwen 2.5 7B / 3B	4.7 / 1.9 GB	Apache-2.0	Ollama (GPU if available)

Model	Size	Licence	Runs on
DreamShaper 8 LCM	2.1 GB	CreativeML OpenRAIL-M (commercial use allowed)	GPU (CUDA) or CPU
all-MiniLM-L6-v2 (ONNX)	90 MB	Apache-2.0	CPU
qwen2.5vl:3b (optional)	3.2 GB	Apache-2.0	Ollama

5. Privacy and trust

Protection	How it works
No uploads	`utils/network_guard.py` refuses any outgoing connection that isn't to this computer.
Local AI only	`llm/ollama.py` refuses AI servers that are not on this computer.
No document text in logs	`utils/logging.py` records only counts and timings.
No invented facts	`intelligence/verification.py` checks every statement against its source; numbers must match exactly. Brand-document condensing (`branddocs/fit.py`) rejects AI text containing numbers that are not in the source.
No invented brand values	`branding/brand_profile.py` and `branding/theme.py` keep unknown values empty and show neutral fall-backs.
Optional online features	GitHub import and licensed image search work only when the user asks. They use an allow-list of hosts and never send document text.

6. Folders

Folder	Contents
`backend/`	The engine (Python)
`frontend/`	The interface (React); `dist/` is the ready-built version
`brands/`	Brand profiles and assets (Zensar)
`scripts/`	Setup, model download, offline bundle, demo recording, developer checks
`samples/`	Example documents, video and papers
`tests/`	Automatic tests

Folder	Contents
`docs/`	This documentation, the slide decks and the demo video
created on first run: `.venv/` `data/` `outputs/` `projects/` `models/` `logs/` `offline/`	Private Python, user data, results, AI models, logs, offline bundle

`docs/User_Manual.pdf` explains every file in plain language.

7. Installation and deployment

- **Normal PC:** double-click `start.bat`. It installs Python, packages, models, the interface and the local AI, then starts the app at http://localhost:8000.
- **GPU server:** `start.bat` detects the NVIDIA GPU and installs the CUDA build of PyTorch. It picks Qwen 2.5 7B when the GPU has 8 GB or more.
- **Corporate proxy:** a proxy configured in Windows or Git is detected and passed on. If pypi.org is still unreachable, everything is installed from the Zen-TMT-offline bundle over Git.
- **Remote access:** `ZCS_HOST=0.0.0.0` (the app has no login, so restrict access with the firewall), or an SSH tunnel.

8. Quality

- **127 automatic tests**: run `python -m pytest`.
- Every conversion ends with a **report**: words preserved, publisher checks, fonts used, images created.
- The documentation in `docs/` is part of the repository and is updated together with the code.

9. Related documents

- `docs/User_Manual.pdf`: installation step by step, how to use every screen, every file explained.
- `docs/Pipeline_Explanation.pdf`: how each pipeline works, step by step.
- `docs/Module_Architecture.pptx`: slides on the modules.
- `docs/Pipeline.pptx`: slides on the pipelines.
- `docs/demo/Zensar_Content_Studio_Demo.mp4`: the demo video.